Assignment 1

CSA0903-Java Programming

Name: Sowmiya. D

Reg No.:192511063

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 5 – Apply static Members and the Object Life Cycle 3 Marks

Bloom's Level: K4 – Analyse

Identify the data and behaviour that belong to the class as a whole rather than to individual objects.

Discuss:

- Where static variables are needed (for example: a ticket counter, total occupancy) and why an instance variable would be wrong
- Where static methods or a static block are appropriate
- What happens to a ParkingTicket object after the vehicle leaves, and when it becomes eligible for garbage collection
- How objects are passed to and returned from methods in your design

Alternative Concept and Complexity Analysis

The ticket counter is a static variable. If it were made an instance variable of ParkingFacility, what would happen when the university runs two facilities (north and south gates)? Propose the solution for each case and analyse the space used by the counter for k facility objects and n tickets.

Evidence Expected

Correct and justified use of static members with an explanation of object passing and the object life cycle.

Solution:

TASK 5 – Apply Static Members and the Object Life Cycle (3 Marks)

Bloom's Level: K4 – Analyse

1. Introduction

In the Campus Smart-Parking System, static members are used to store data and perform operations that belong to the entire class rather than individual objects.

For example, a ticket counter and total parking occupancy can be maintained using static variables. Static members help share common information among all objects.

2. Static Variables

Static variables are declared using the static keyword. They are shared by all objects of a class.

Static variable	**Purpose**
ticketCounter	Generates unique ticket numbers.
totalOccupancy	Tracks the total number of occupied slots.
totalTicketsIssued	Counts the tickets issued by the system.

Example:

```
class ParkingTicket {
    static int ticketCounter = 1000;
    int ticketId;
    String vehicleNumber;
    ParkingTicket(String vehicleNumber) {
        ticketId = ++ticketCounter;
        this.vehicleNumber = vehicleNumber;
    }
}
```

Why is a static variable needed?

The ticket counter must be shared by all ParkingTicket objects. If it were an instance variable, every ticket object would maintain a separate counter, which could generate duplicate ticket numbers.

3. Static Methods and Static Blocks

A. Static Method

A static method belongs to the class and can be called without creating an object.

Example:

```
class ParkingFacility {
    static int totalOccupancy = 0;

    static void displayOccupancy() {
        System.out.println("Total Occupancy: " + totalOccupancy);
    }
}
```

Usage:

```
ParkingFacility.displayOccupancy();
```

B. Static Block

A static block is executed only once when the class is loaded. It is used to initialize static variables.

Example:

```
class ParkingTicket {
    static int ticketCounter;

    static {
        ticketCounter = 1000;
    }
}
```

Uses:

- Static methods display common parking information.
- Static blocks initialize shared values.

- They avoid unnecessary object creation for class-level operations.

4. Object Life Cycle of a ParkingTicket

A ParkingTicket object is created when a vehicle enters the parking facility.

The object stores:

- Unique ticket number.
- Vehicle registration number.
- Entry time.
- Exit time.
- Parking fee.

When the vehicle leaves, the ticket is closed, the parking slot is released and the fee is calculated.

After the ticket is no longer referenced by the program, the object becomes eligible for garbage collection.

Example:

```
ParkingTicket ticket = new ParkingTicket("TN01AB1234");


// Vehicle leaves

ticket = null;
```

Here, ticket = null removes the reference to the object. If no other references exist, the object becomes eligible for garbage collection.

Garbage Collection: Java automatically identifies and reclaims memory occupied by objects that are no longer reachable.

5. Passing and Returning Objects from Methods

In Java, objects can be passed as arguments to methods and returned from methods.

Example:

```
class ParkingService {


   void displayTicket(ParkingTicket ticket) {

      System.out.println(ticket.vehicleNumber);

   }
```

```
    ParkingTicket createTicket(String number) {
        return new ParkingTicket(number);
    }
}
```

Explanation:

- displayTicket() receives a ParkingTicket object as an argument.
- createTicket() creates and returns a ParkingTicket object.
- Object references allow different classes to work together.

6. Alternative Concept and Complexity Analysis

The ticket counter is maintained as a static variable in the original design.

Suppose the university has two parking facilities: North Gate and South Gate.

Alternative: Using an Instance Variable

If ticketCounter is an instance variable of ParkingFacility, each facility maintains its own counter.

Example:

```
class ParkingFacility {
    int ticketCounter = 1000;
}
ParkingFacility north = new ParkingFacility();
ParkingFacility south = new ParkingFacility();
```

Both facilities start with the same counter value. Therefore, duplicate ticket numbers may be generated.

Solution 1: Static Counter

```
class ParkingTicket {
    static int ticketCounter = 1000;
}
```

Both facilities share the same counter, so ticket numbers are generated from a common sequence.

Solution 2: Separate Counter for Each Facility

Each facility can maintain its own counter with a unique facility prefix.

For example:

- North Gate: N-1001
- South Gate: S-1001

This avoids duplicate ticket numbers between facilities.

Comparison

Criteria	Static counter	Instance counter
Storage	Shared by all objects	Separate for each facility
Counter values	Common sequence	Independent sequence
Duplicate risk	Lower with shared counter	Possible across facilities
Memory usage	One counter	One counter per facility

Time and Space Complexity

Let:

- k = Number of parking facility objects.
- n = Number of parking tickets.

Criteria	Static counter	Instance counter
Ticket generation	O(1)	O(1)
Creating n tickets	O(n)	O(n)
Counter space	O(1)	O(k)
Ticket storage	O(n)	O(n)

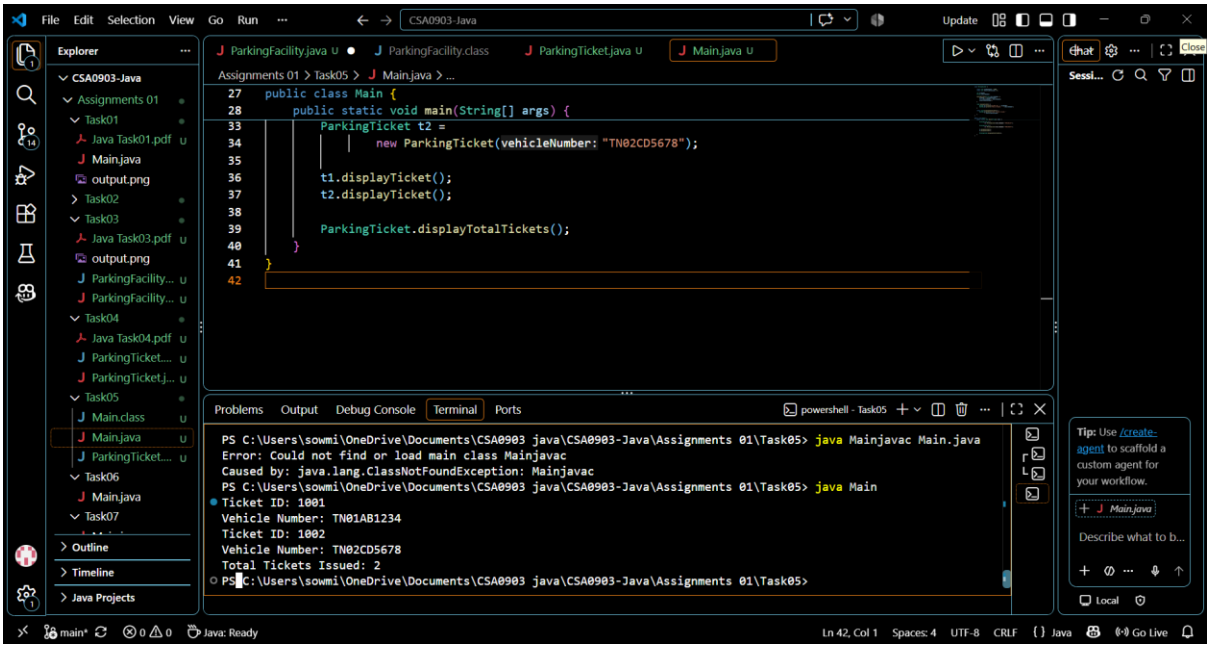
The static counter requires constant space because only one shared counter is maintained. An instance counter requires additional space for every facility object.

7. Conclusion

Static members are useful for maintaining common information such as ticket numbers and total occupancy in the Campus Smart-Parking System. Static methods perform class-level operations, while static blocks initialize shared data.

When a vehicle leaves, its ParkingTicket object becomes eligible for garbage collection once it is no longer reachable.

Recommended approach: Use a static ticket counter to generate unique ticket numbers across multiple parking facilities. This reduces memory usage and avoids duplicate ticket numbers within the shared system.


File Edit Selection View Go Run
CSA0903-Java
Update
Explorer
CSA0903-Java
Assignments 01
Task01
Java Task01.pdf
Main.java
output.png
Task02
Task03
Java Task03.pdf
output.png
ParkingFacility...
ParkingFacility...
Task04
Java Task04.pdf
ParkingTicket....
ParkingTicket.j...
Task05
Main.class
Main.java
ParkingTicket....
Task06
Main.java
Task07
Outline
Timeline
Java Projects
ParkingFacility.java U
ParkingFacility.class
ParkingTicket.java U
Main.java U
Assignments 01 > Task05 > Main.java > ...
27 public class Main {
28 public static void main(String[] args) {
33 ParkingTicket t2 =
34 new ParkingTicket(vehicleNumber: "TN02CD5678");
35
36 t1.displayTicket();
37 t2.displayTicket();
38
39 ParkingTicket.displayTotalTickets();
40 }
41 }
42
Problems Output Debug Console Terminal Ports
powershell - Task05
PS C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task05> java Mainjavac Main.java
Error: Could not find or load main class Mainjavac
Caused by: java.lang.ClassNotFoundException: Mainjavac
PS C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task05> java Main
Ticket ID: 1001
Vehicle Number: TN01AB1234
Ticket ID: 1002
Vehicle Number: TN02CD5678
Total Tickets Issued: 2
PS C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task05>
chat
Close
Sessi...
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Main.java
Describe what to b...
Local
main*
0 0
Java: Ready
Ln 42, Col 1
Spaces: 4
UTF-8
CRLF
Java
Go Live